DEPARTMENT OF
NETWORKED SYSTEMS
AND SERVICES

Szoftverbiztonság

VIHIAC07 – IT biztonság, 2026

Dr. András Gazdag

CrySyS Lab, BME

andras.gazdag@crysys.hu

www.crysys.hu

MŰEGYETEM 1782

Tartalom

- A szoftverbiztonság helyzete manapság
- Security Development Lifecycle
  - Pre-SDL: tanulás
  - SDL: követelmények, tervezés, implementáció, tesztelés, kiadás
  - Post-SDL: üzemeltetés feladatai

Szoftverbiztonság helyzete

Common Vulnerabilities and Exposures

- CVE: nyilvános adatbázis a megtalált sérülékenységekről

CVE-ID	
CVE-2017-16530	Learn more at National Vulnerability Database (NVD) • CVSS Severity Rating • Fix Information • Vulnerable Software Versions • SCAP Mappings • CPE Information
Description	
The uas driver in the Linux kernel before 4.13.6 allows local users to cause a denial of service (out-of-bounds read and system crash) or possibly have unspecified other impact via a crafted USB device, related to drivers/usb/storage/uas-detect.h and drivers/usb/storage/uas.c.	
References	
Note: References are provided for the convenience of the reader to help distinguish between vulnerabilities. The list is not intended to be complete.	
• MISC:https://github.com/torvalds/linux/commit/786de92b3cb26012d3d0f00ee37adf14527f35c4 • MISC:https://groups.google.com/d/msg/syzkaller/pCswO77gRlM/VHuPOftgAwAJ	
Assigning CNA	
MITRE Corporation	
Date Entry Created	
20171103	Disclaimer: The entry creation date may reflect when the CVE ID was allocated or reserved, and does not necessarily indicate when this vulnerability was discovered, shared with the affected vendor, publicly disclosed, or updated in CVE.
Phase (Legacy)	
Assigned (20171103)	

Apple product security notifications

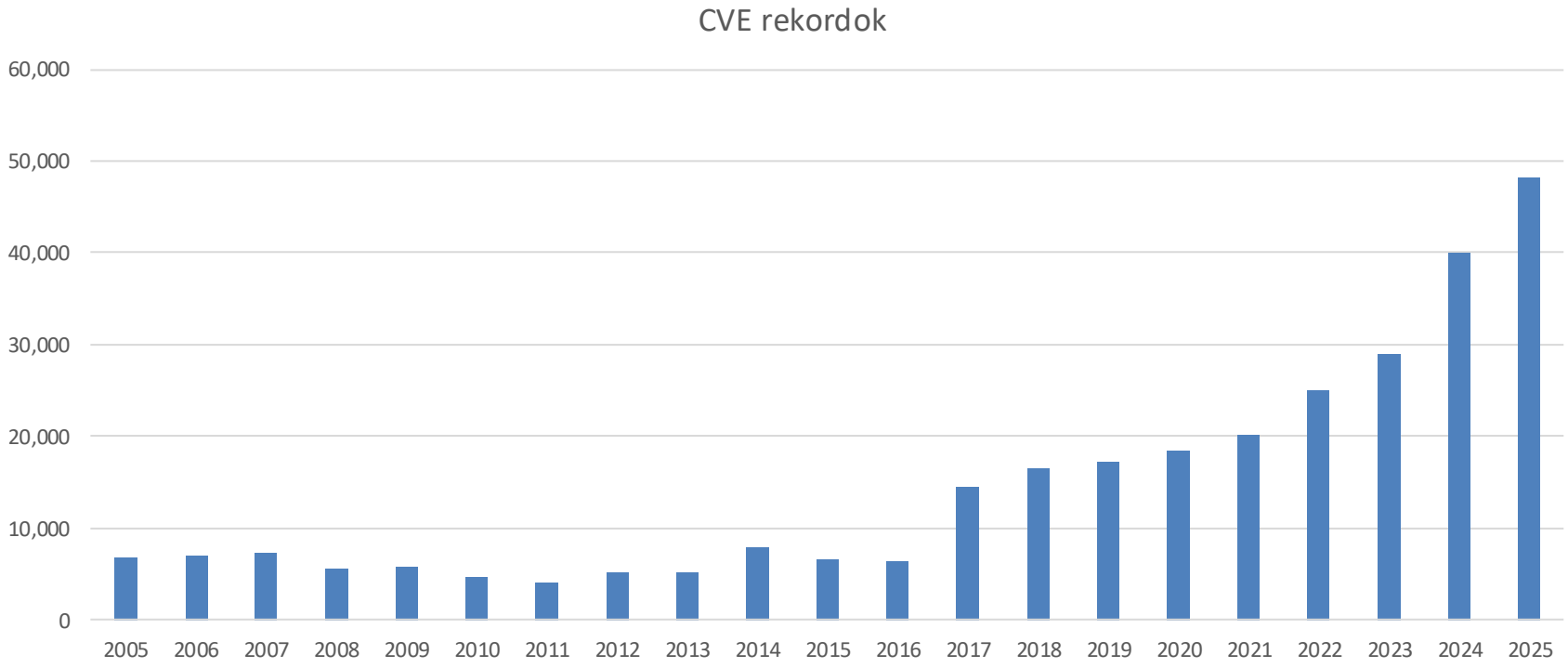
WebKit
Available for: macOS Ventura and macOS Sonoma
Impact: Processing maliciously crafted web content may lead to an unexpected process crash
Description: This issue was addressed through improved state management.
WebKit Bugzilla: 284159
CVE-2025-24162: linjy of HKUS3Lab and chluo of WHUSecLab

CoreMedia
Available for: macOS Sequoia
Impact: A malicious application may be able to elevate privileges. Apple is aware of a report that this issue may have been actively exploited against versions of iOS before iOS 17.2.
Description: A use after free issue was addressed with improved memory management.
CVE-2025-24085

Contacts
Available for: macOS Ventura
Impact: An app may be able to access contacts
Description: A privacy issue was addressed with improved private data redaction for log entries.
CVE-2024-44172: Kirin (@Pwnrin)

Common Vulnerabilities and Exposures

- CVE: nyilvánosan elérhető sérülékenységadatbázis



- Átlagosan 12 percenként bekerül egy új rekord (!)

Mik a kihívások?

1. Lejt a pálya
   - Fejlesztői megkötések: idő, erőforrások, funkcionalitás
   - Támadói megkötések: motiváció és felkészültség

2. Biztonsági tesztelés maga
   - Funkcionális tesztelés: hogyan kéne működnie a rendszernek
   - Biztonsági tesztelés: hogyan NEM kéne működnie a rendszernek

3. Gyenge üzleti motiváció
   - Nehéz mérni a biztonságot → a vevők nem kényszerítik ki a versenyt

4. Végfelhasználót éri kár
   - A fejlesztők nem elég motiváltak

Akkor miért is vagyunk itt?

- Megnyerhetetlen harcnak tűnik
  - Ami hatalmas befektetést igényel
  - Miközben nem tudjuk tisztán mérni, hogy mennyire végeztünk jó munkát
- Az incidensek 90% ugyan azokra az okokra vezethetők vissza
  - Ezeket az okokat ismerjük és értjük!
    - » Hatásos, olcsó és célzott védekezési mechanizmusok
- Elsőre jól csinálni nem kerül semmibe ☺

Fejlesztési életciklus

Szoftverfejlesztési életciklusok

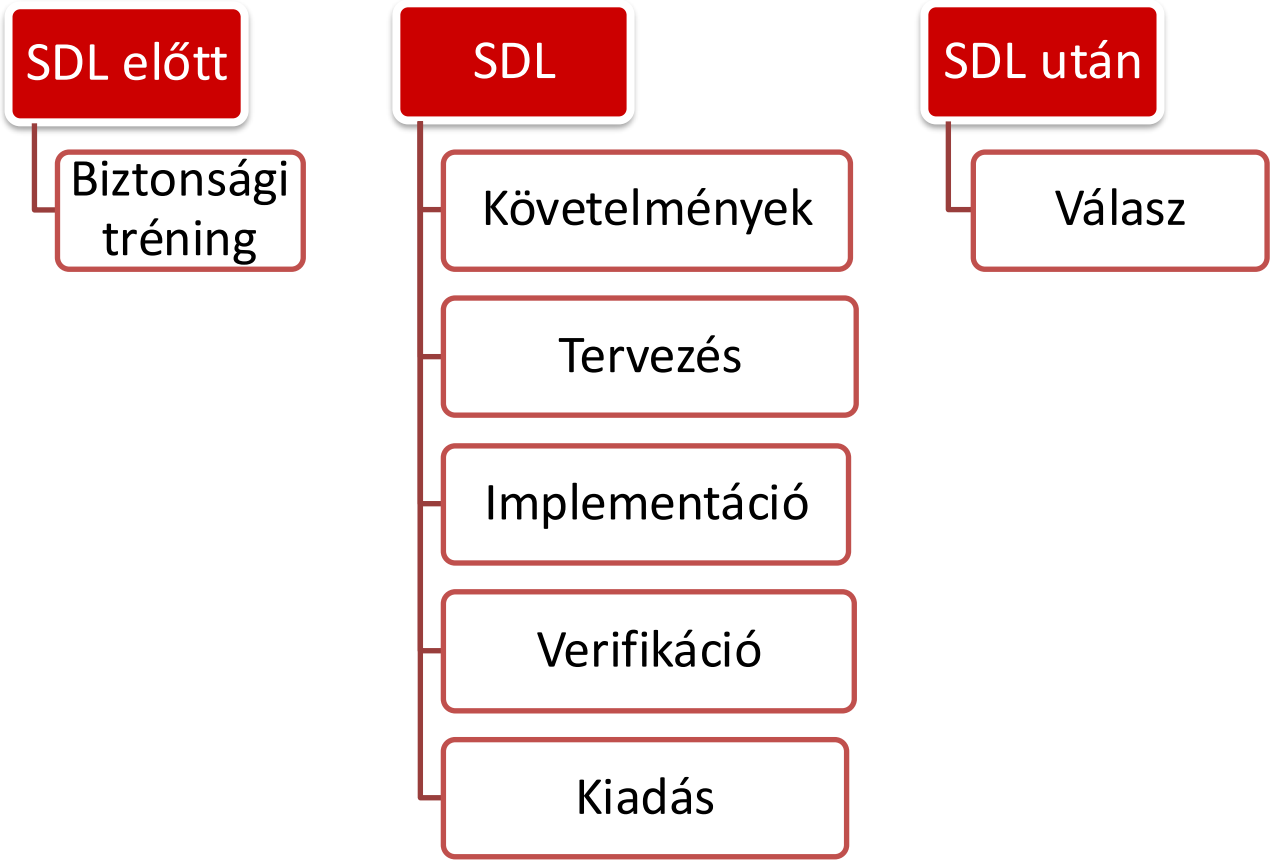
- A szoftverfejlesztés egy folyamat, ami sokféle tevékenységből áll
- Hogyan menedzseljük ezeket a tevékenységeket?
- Életciklus modellek
  - Szekvenciális: vízesés, V-modell
  - Inkrementális, iteratív, spirális
  - Agilis metodológiák: scrum, extreme programming, lean, stb.

Szoftverfejlesztési életciklusok

- Biztonsággal kapcsolatos tevékenységek nem jelennek meg!
- Biztonságos szoftverfejlesztési életciklus modellek
  - Teljesen átalakított folyamat
    - » OWASP CLASP
    - » **Microsoft SDL**
    - » Software Security Touchpoints
  - Integrálandó tevékenységek listája
    - » OpenSAMM
    - » BSIMM
    - » SAFECode

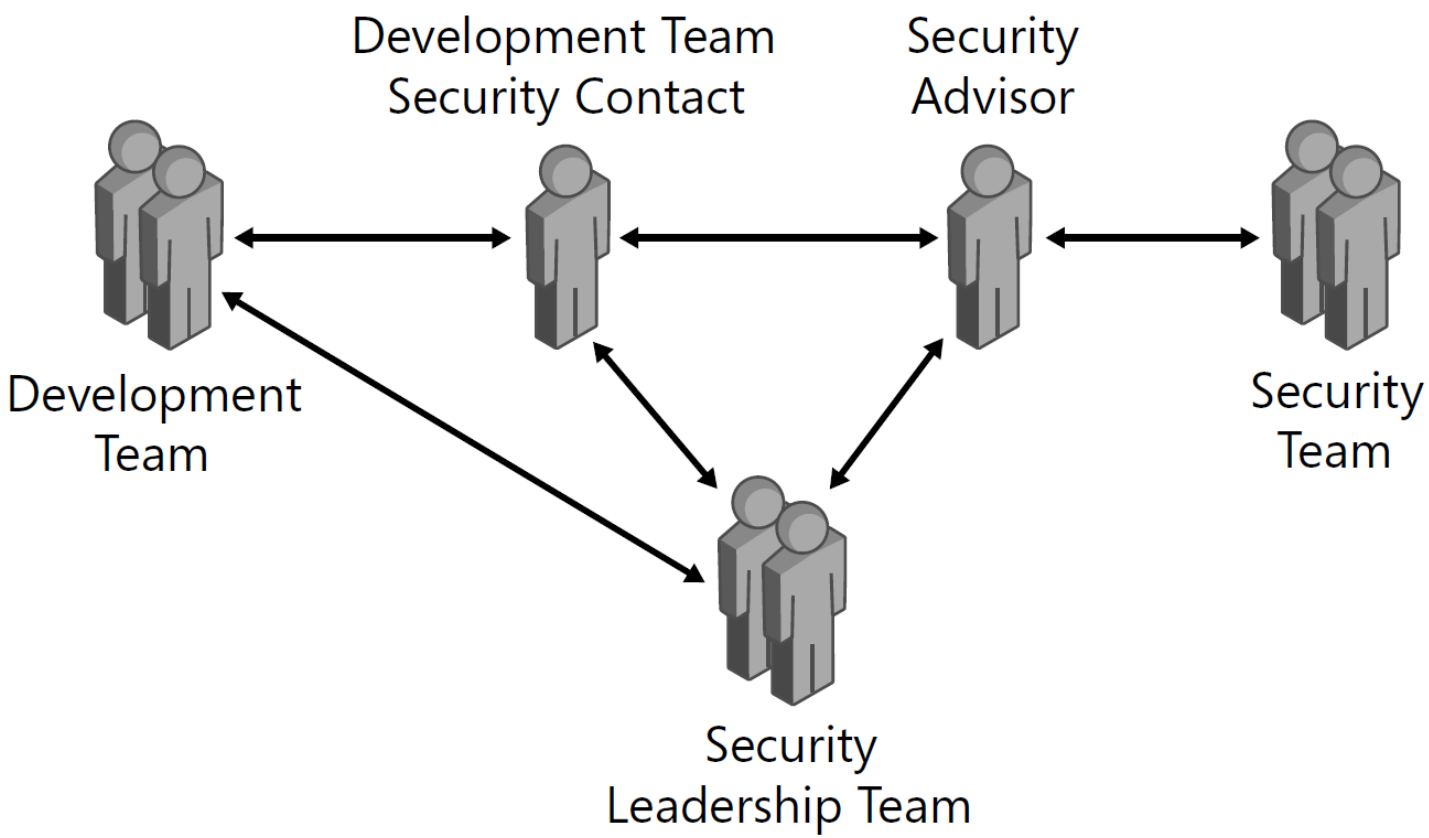
Microsoft SDL: Áttekintés

- Csökkentsük a sérülékenységek számát fejlesztés közben
- Csökkentsük a meg nem talált biztonsági bugok súlyosságát



Áttekintés: egy csapat felépítése

Csapat felépítése: biztonság, mint szempont, képviselete



Követelmények

Fejlesztési életciklus

SDL: Követelmények

- Cél: a funkcionális követelmények mellett a biztonsági követelmények definiálása
  - CIA + 3A

- Threat, Risk, Vulnerability Analysis
  - Cél: dokumentáljuk a biztonsági mechanizmusok beépítésének indokait
    → Azonosított veszélyforrások és biztonsági/privacy követelmények
    → Szükséges biztonsági funkcionalitások listája
  - Általában brainstromingoló megbeszélések keretei között készül
  - Ha a rendszer változik, újra kell csinálni!

Threat, Risk, Vulnerability Analysis

1. A rendszer határainak a meghatározása
2. A biztonsági követelmények és célok definiálása
3. Fenyegetések összegzése
4. Kockázatok vizsgálata
5. Védelmi megoldások azonosítása

- Eredmény: a fejlesztendő rendszer specifikációja

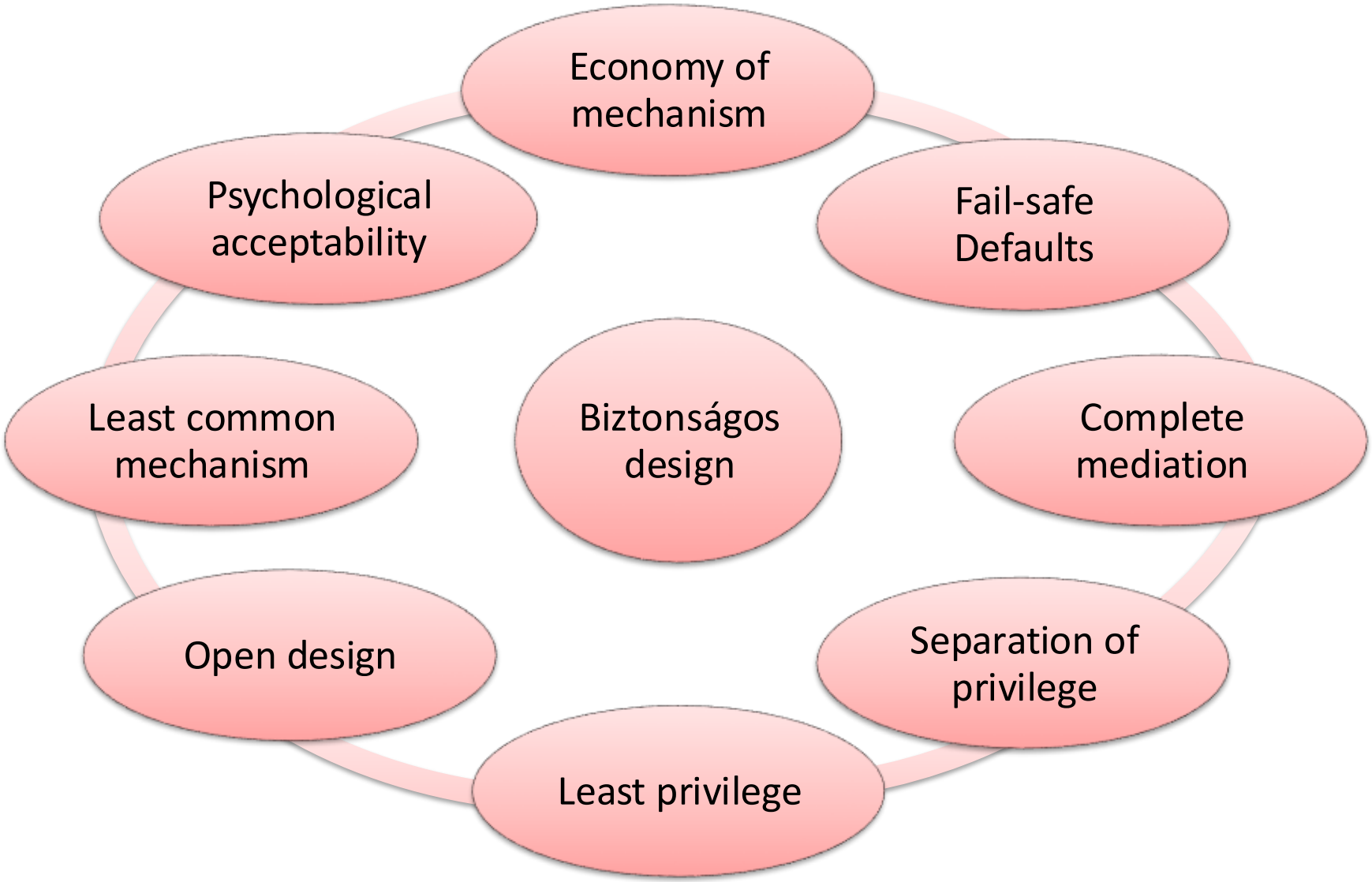
Tervezés

Fejlesztési életciklus

SDL: Tervezés – Általános elvek

- Mélységi védelem (Defense in Depth)
  - Több biztonsági védelmi réteg alkalmazása, hogy egy esetleges biztonsági incidens esetén is legyen tartalék védelmi mechanizmus.
- Frissíthetőségre tervezés (Design for Updating)
  - A szoftver biztonságát úgy kell megtervezni, hogy az könnyen módosítható legyen, például biztonsági javítócsomagok (patchek) és biztonsági tulajdonságok változtatása esetén.
- Shift Left on Security
  - A biztonsági auditok, fenyegetésmodellezés és tesztelés integrálása a fejlesztési életciklus korai fázisaiba, ne utólagos megoldásként kezelje a csapat.
- Fenyegetésmodellezés (Threat Modeling)
  - Annak szisztematikus feltérképezése, hogy mi mehet rosszul – még az implementáció megkezdése előtt, ne azt követően.

SDL: Tervezés fázis – Tervezési alapelvek



SDL: Tervezés fázis – Open Design

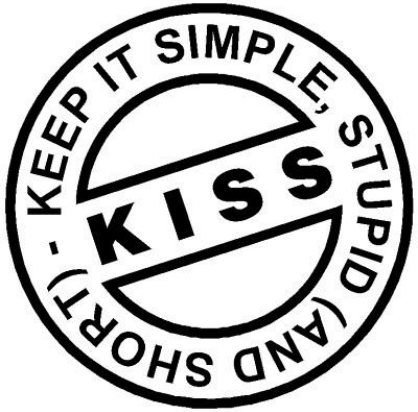
- Ne bízz a tervek titkosságában
  - Kapcsolódó fogalom: security by obscurity
- Egy rendszer terveit érdemes elérhetővé tenni, hogy minél szélesebb körben elemezhessék
  - Több szem többet lát!
  - Open source világ előnye
  - Ezt a hozzáállást követi a kripto protokollok világa

SDL: Tervezés fázis – KISS

- Minél komplexebb a szoftver, annál valószínűbbek a bugok
- Kisebb kódbázist könnyebb karbantartani

- Amerikai haditengerészettől ered
- Kelly Johnson repülőgép-mérnökhöz kötik

- A „kicsit” sose az egyszerűség rovására érjük el!



SDL: Tervezés fázis – Least Common Mech.

- Minimalizáljuk azon mechanizmusok számát, amik
  1. Több felhasználó számára is elérhetőek, és
  2. Minden felhasználó számára szükségesek

- A megosztott mechanizmus egy potenciális információátviteli csatorna
- Különböző mechanizmusok (akár különböző instance-ek!) nagyobb rugalmasságot nyújtanak

- Példa: folyamatok memóriájának szeparációja

SDL: Tervezés fázis – Fail-Safe Defaults

- Mi van, ha valami nem a várakozásoknak megfelelően történik?

- Tiltólista
  - Alapértelmezés: hozzáférés **engedélyezve**
  - Ha hibázunk: fals pozitív döntések
    - » A hozzáférést megadjuk, pedig nem kellett volna
    - » **A felhasználó nem fogja jelezni!**

- Engedélyezőlista
  - Alapértelmezés: hozzáférés **megtagadva**
  - Kivéve, ha valamilyen szabály engedi
  - Ha hibázunk: fals negatív döntések
    - » Megtagadjuk a hozzáférést, pedig engedni kellett volna
    - » **Ezt jelezni fogja a felhasználó ☺**

SDL: Tervezés fázis – Complete Mediation

- Ellenőrizzünk minden elérést minden objektumhoz
  - Szkepticizmus: meg kéne engednem?

- Sok rendszer cache-eli a „hozzáférés engedélyezve” eredményt, majd a cache-t használja adott ideig
  - Mi van, ha közben visszavonták az engedélyt??

SDL: Tervezés fázis – Least Privilege

- Csak annyi jogosultságot használjunk, ami feltétlenül szükséges a feladat elvégzéséhez
- → Kisebb kárt okoznak a balesetek és hibák
  - Kódot szúrt be a támadó → a fertőzött program jogosultságaival fog futni
- Magasabb jogosultságot csak időlegesen szerezzünk!
- Példák: hadászati ”need-to-know”, sandboxok

SDL: Tervezés fázis – Separation of Priv.

- Több feltételnek is kell teljesülnie, mielőtt hozzáférést adunk
  - Robusztusabb és rugalmasabb rendszer
  - Egyetlen ellenőrzés: kijátszható, kudarcot vallhat

- Előfeltétel: részekre osztás
  - Bontsuk fel a rendszert kisebb egységekre
  - → Minden egység más-más feltételt ellenőrizhet

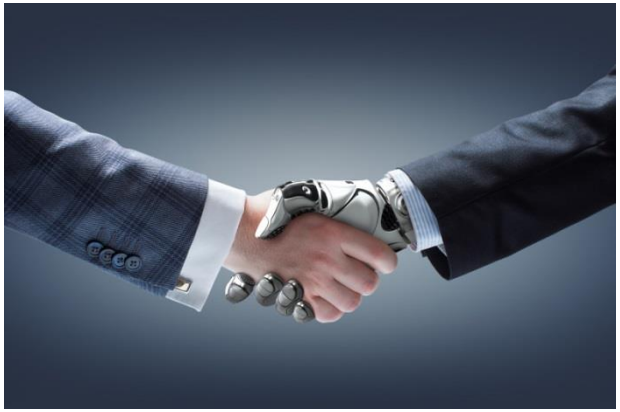
- Példa: kétfaktoros bejelentkezés

SDL: Tervezés fázis – Psychological Accept.

- Ha a felhasználók nem fogadják el, meg fogják próbálni kikerülni
- Figyeljünk az emberi tényezőre!

- A rendszer használata legyen egyszerű és intuitív
- Az erőforrás-hozzáférés maradjon egyszerű

- Példa: ssh login privát-publikus kulcspárral



Implementáció

Fejlesztési életciklus

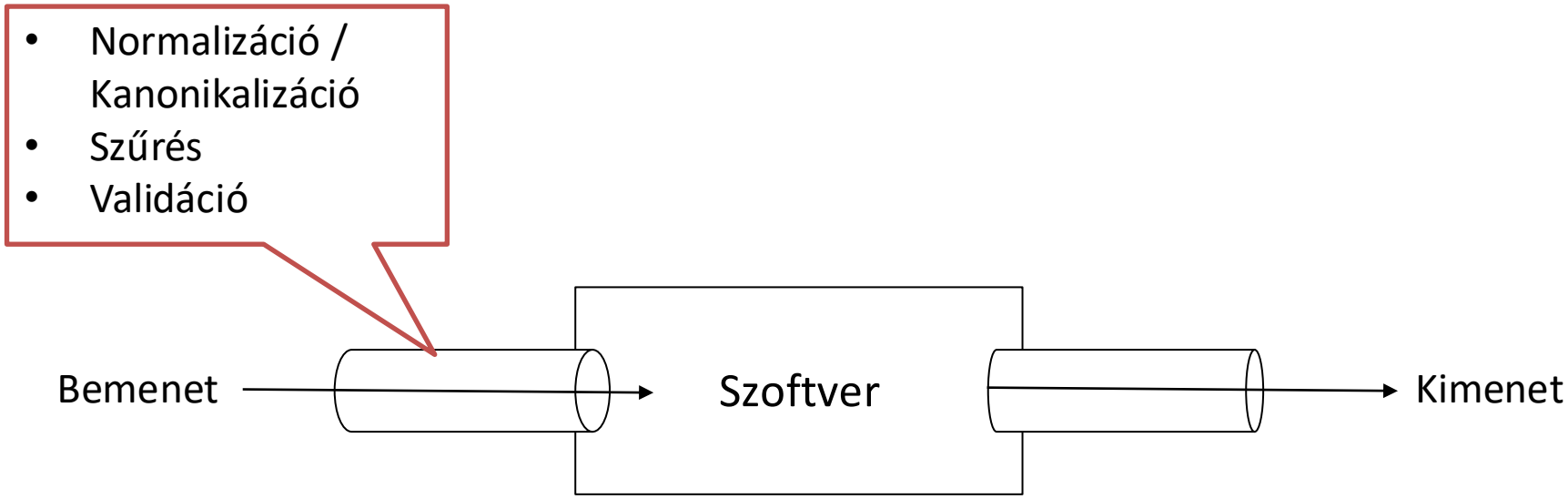
SDL: Implementáció fázis

- Kezdjünk kódolni!
- Ugyan mi sikerülhet félre?
  - Input validáció
  - Hibakezelés
  - Naplózás
  - Versenyhelyzet
  - Stb.

- Vegyük figyelembe a szoftver **támadási** felületét!
  - Minden végrehajtási út, min keresztül adat/parancs érkezik/távozik
    - » Plusz a kód, ami ezeket a végrehajtási utakat védi
  - Minden értékes adat, amit a szoftver használ
    - » Plusz a kód, ami ezeket védi

SDL: Implementáció fázis – Adatfeldolgozás

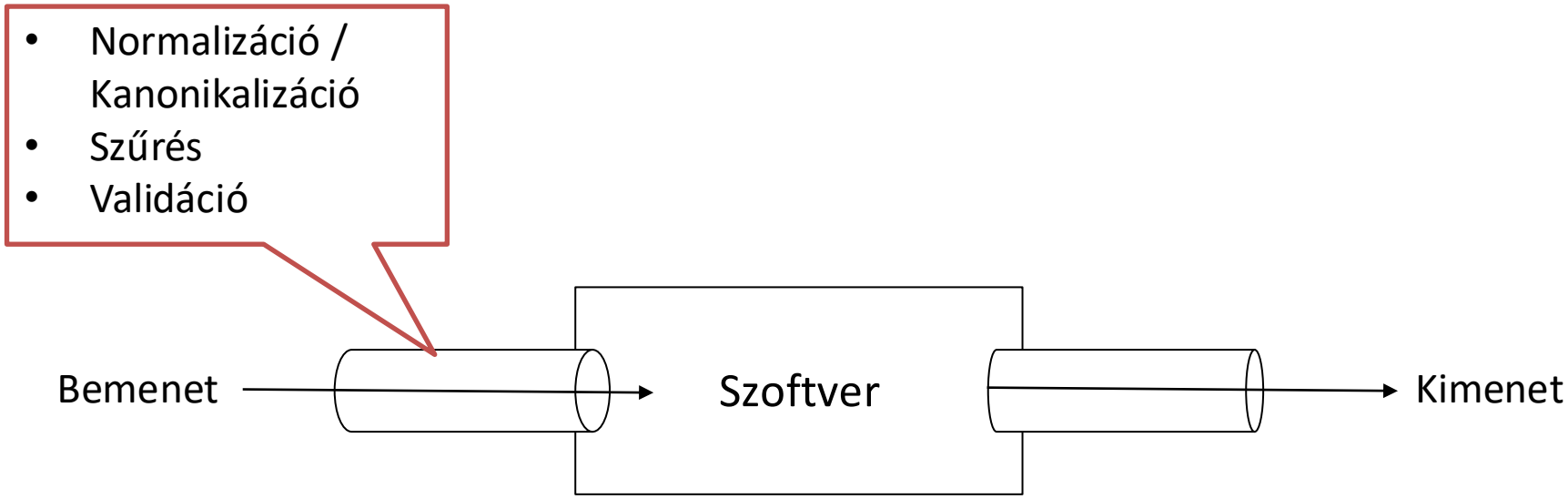
- Bemenetet a felhasználó kontrollálja → **megbízhatatlan**



- Normalizáció / kanonikalizáció: átalakítás a legegyszerűbb (és elvárt) formára
  - Pl. egy dátum egységes formátumra hozása: YYYY-mm-dd

SDL: Implementáció fázis – Adatfeldolgozás

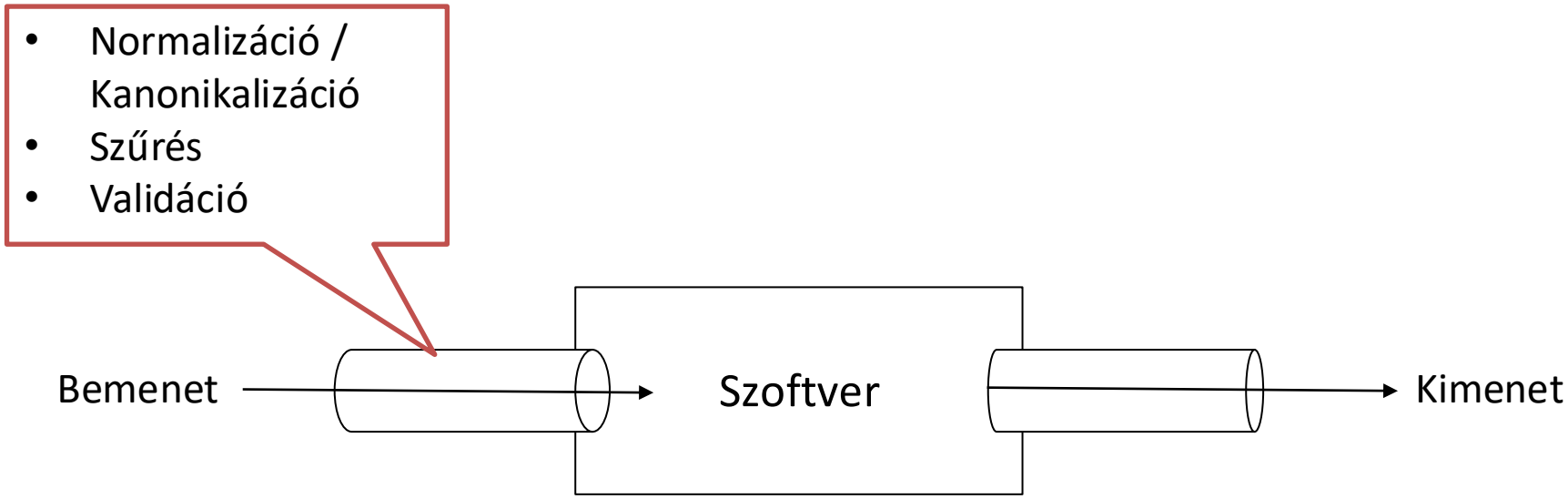
- Bemenetet a felhasználó kontrollálja → **megbízhatatlan**



- Szűrés: bizonyos kritériumok alapján egyes elemek eltávolítása a bementből (nem feltétlen biztonsági okból)
  - Pl. egy dátumban a hónap nem lehet 12-nél nagyobb

SDL: Implementáció fázis – Adatfeldolgozás

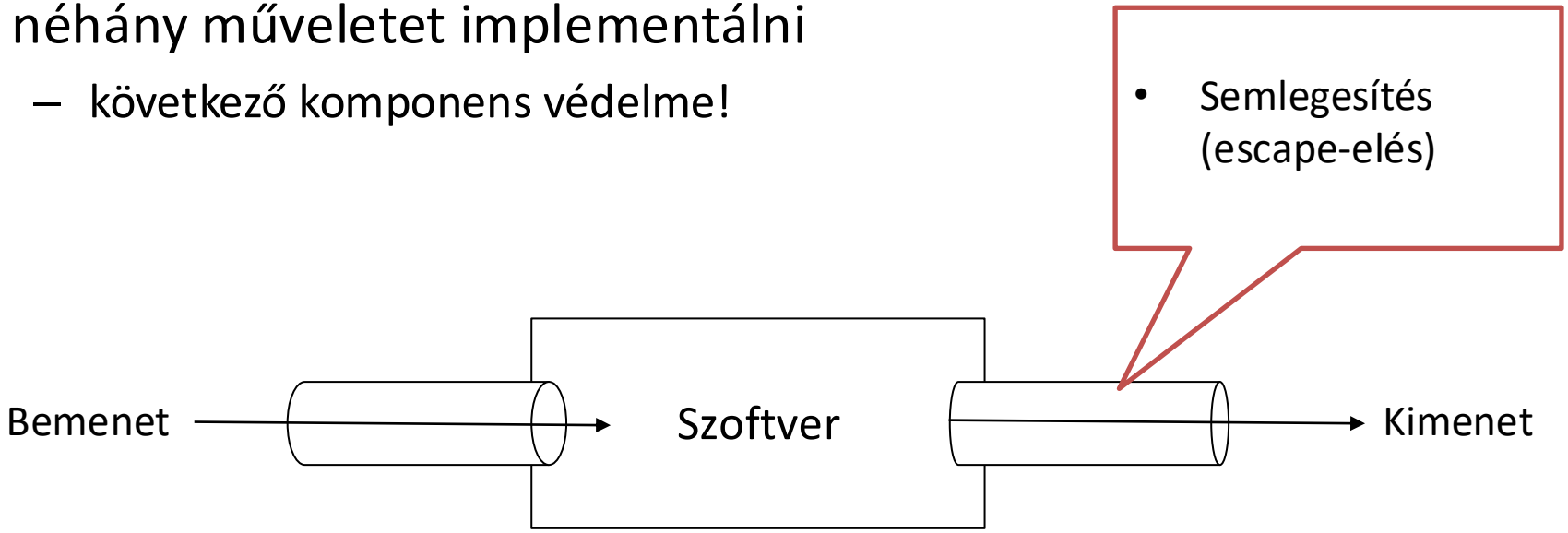
- Bemenetet a felhasználó kontrollálja → **megbízhatatlan**



- Validáció: az bemenet ésszerűségének ellenőrzése (ide értve létezést, típust, korlátokat, konzisztenciát, stb.)
  - Pl. egy születési dátum ne legyen túl régi vagy túl új

SDL: Implementáció fázis – Adatfeldolgozás

- Kimenetet más komponens felhasználhatja, így ott is kell néhány műveletet implementálni
  - következő komponens védelme!



- Semlegesítés (escape-elés): valamilyen biztonsági kritériumnak megfelelő kimenet előállítása, amiben nem fordulhatnak így elő káros elemek

SDL: Implementáció fázis – Adatfeldolgozás

- Példa: input validációs mérés

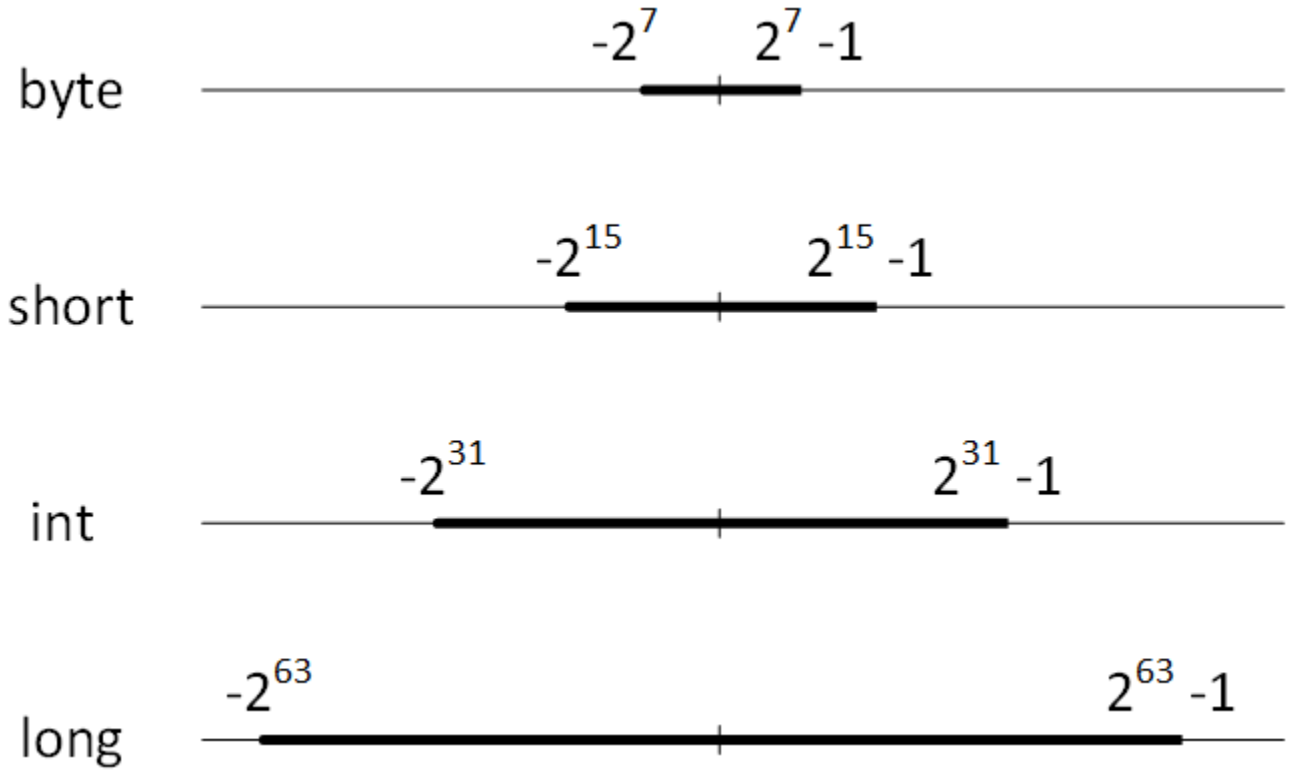
```python
# read the width
width = ciff_file.read(8)
# check if width contains 8 bytes
if len(width) != 8:
    raise Exception("Invalid image: incorrect width size")

# interpret the bytes as an 8-byte-long integer
new_ciff.width = struct.unpack("Q", width)[0]

# the width must be in [0, 2^64 - 1]
if new_ciff.width < 0 or new_ciff.width > (2**64)-1:
    raise Exception("Invalid image: incorrect width value")

# the width must be in [0, 4 * 2^10]
if new_ciff.width < 0 or new_ciff.width > 4*(2**10):
    raise Exception("Image too big to be displayed")
```

SDL: Implementáció fázis – Adatfeldolgozás



- Mi történik az egyes tartományok határán?

SDL: Implementáció fázis – Adatfeldolgozás

```
1 class Example
2 {
3     static int myAdd(int a, int b) {
4         return a+b;
5     }
6 }
```

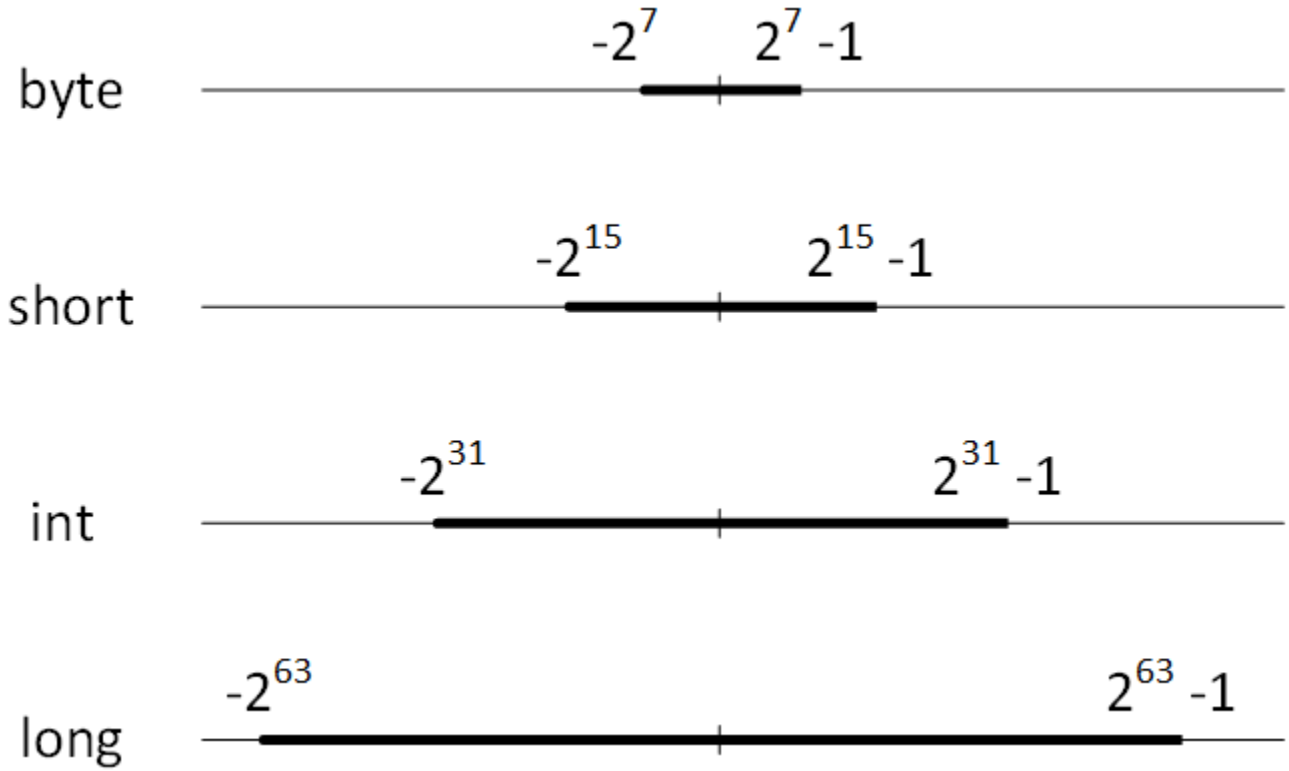
SDL: Implementáció fázis – Adatfeldolgozás

```
1 class Example
2 {
3     static int myAdd(int a, int b) {
4         return a+b;
5     }
6 }
```

- a = Integer.MAX_VALUE
- b = 1

- → a+b = Integer.MIN_VALUE ☺

SDL: Implementáció fázis – Adatfeldolgozás



- Mi történik az egyes tartományok határán?
  - *Nincs arra garancia, hogy egy aritmetikai művelet eredménye matematikailag is helyes lesz!*

SDL: Implementáció fázis – Hibakezelés

- A nem megfelelő hibakezelés biztonsági sérülékenységekhez vezethet

- Gyakori hibák:
  - Túlságosan tág hibaüzenet és/vagy hibakezelés
  - Hiba (Error) vs. Kivétel (Exception) – melyiket használjuk?
  - Érvényes állapot visszaállításának hiánya kivétel után
  - Hibakezelés részleges vagy teljes hiánya
  - Információszivárgás

SDL: Implementáció fázis – Hibakezelés

```
1 try {
2     doOperation();
3 }
4 catch(Exception e) {
5     // this can never happen
6 }
```

- Híres utolsó mondatok…
- Mi itt a valódi hiba, amit a fejlesztő elkövetett?

SDL: Implementáció fázis – Naplózás

- A naplófájlok fontos adatforrások:
  - Biztonsági incidensek azonosítása
  - Szabályszegések monitorozása
  - Letagadhatatlansági mechanizmusok támogatása
  - Incidensek kivizsgálása

- Hogyan készítsünk naplóbejegyzéseket?
  - *Mikor*: naplóbejegyzés dátuma és ideje, esemény dátuma és ideje
  - *Hol*: komponens azonosítók, címek, kódban hely, geolokáció, szolgáltatás
  - *Ki*: IP cím, felhasználói azonosító (felhasználónév, licensz száma, …)
  - *Mit*: esemény típusa, súlyossága, leírása, biztonsággal kapcsolatos-e?

SDL: Implementáció fázis – Naplózás

- Milyen eseményeket naplózzunk mindenképp?
  - Adatfeldolgozási hibák
  - Autentikáció eredménye (sikeres vagy sem)
  - Autorizációs hiba
  - Session menedzsment
  - Hibák és rendszeresemények
  - Naplózás kezdete
- Mik NE szerepeljenek a bejegyzésekben?
  - Kulcsok, jelszavak, tokenek
  - Forráskód
  - Más érzékeny információk
- Helyes hozzáállás: naplófájlokat bárki olvashatja!

SDL: Implementáció fázis – Naplózás

```
1 if (loginSuccessful) {
2     logfile.write("User login succeeded for: " + username);
3  } else {
4     logfile.write("User login failed for: " + username);
5 }
```

- Itt sem szabad elfelejteni az input validációt!

- Input: `user\nUser login succeeded for: admin`
- Ennek a hatása:

„`User login succeeded for: user`
`User login succeeded for: admin`" ← Ez meg ki volt??

- Érdemes standard naplózási megoldásokat használni, mint pl a syslog!

Tesztelés

Fejlesztési életciklus

SDL: Tesztelés fázis

- Cél: ellenőrizzük a CIA hármast + 3A-t
- Minden életciklus fázisban kell tesztelni!

- „Program testing can be a very effective way to show the presence of bugs, but it is hopelessly inadequate for showing their absence.” – Edsger W. Dijkstra

SDL: Tesztelés fázis – Statikus elemzés

- Az utasításokat csak értelmezzük, nem hajtjuk végre
- Az elemzés egy absztrakt domén fölött történik automatizáltan vagy manuálisan

- Előnyök
  - Skálázhatóság: nagy kódbázist is képes kezelni
  - A fejlesztés során korán el lehet kezdeni alkalmazni

- Hátrány
  - Nincs futásidejű információ→ fals pozitív eredmény: olyan utasításokat is sérülékenynek jelezhet, amit a valóságban sose lehet lefuttatni

SDL: Tesztelés fázis – Dinamikus elemzés

- A szoftver futtatás közben elemezzük
- Előnyök
  - Elérhetők futási idejű információk→ pontosabb eredmények
  - Jelzett sérülékenységek biztosan benne vannak a szoftverben
- Hátrányok
  - Instrumentációra lehet szükség (fordított kódra nem triviális!)
  - Alacsony kód-lefedettség
  - Nem tud semmit mondani olyan viselkedésről, amit nem figyelt meg

SDL: Tesztelés fázis – További módszerek

- Fuzzing
  - Automatikus generált véletlenszerű bemenetek
  - Szoftvert futás közben monitorozzuk hibákat keresve
- Sérülékenység vizsgálat
  - Automatizált folyamat sérülékenységek kereséséhez
  - Csak jól ismert sérülékenységeket talál meg
- Penetrációs tesztelés
  - Futó rendszeren (teszt vagy production) végzett elemzés
  - Cél: demonstrálni, hogy mit tehet egy támadó

Kiadás

Fejlesztési életciklus

SDL: Kiadás fázis

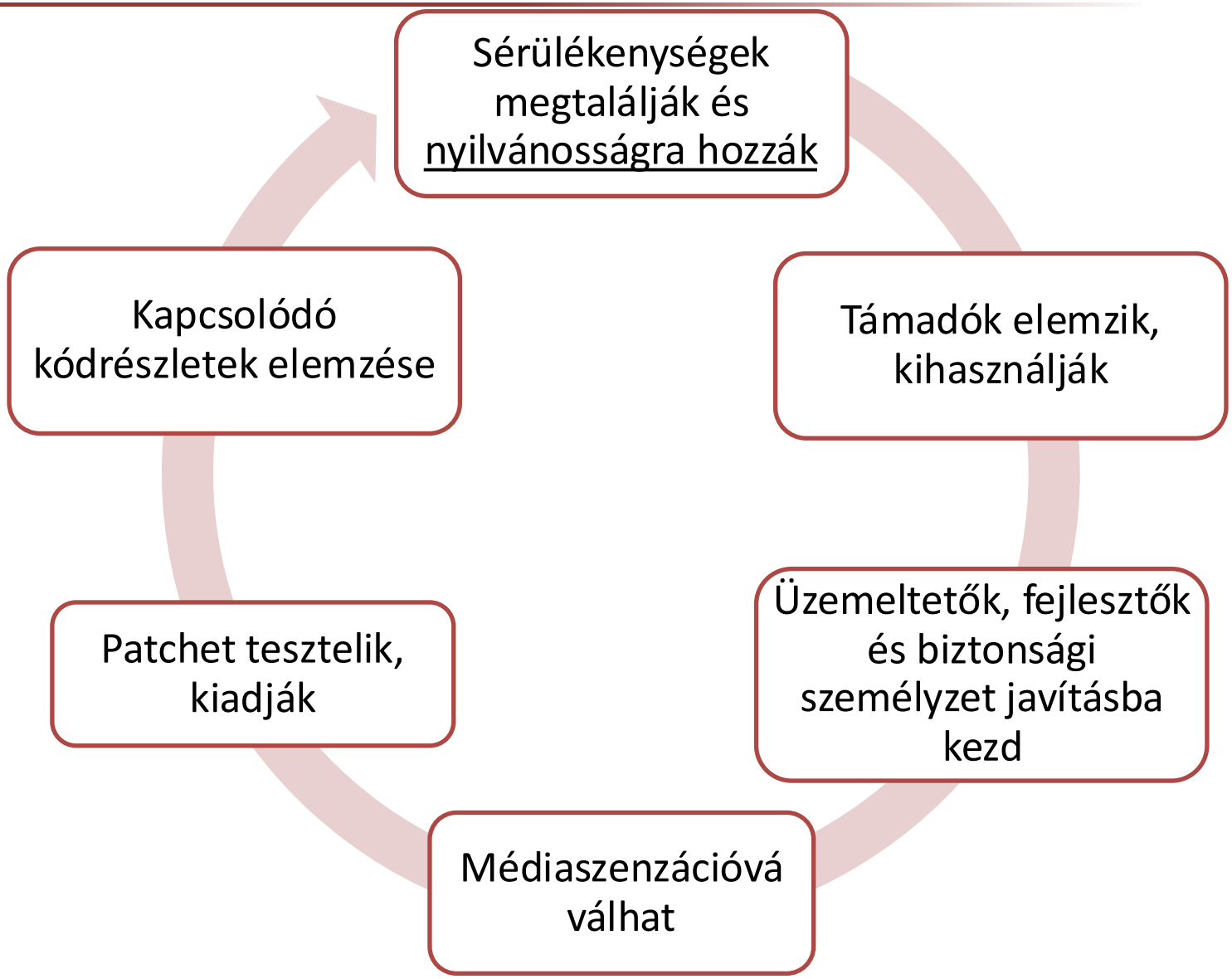
- **Security response terv** → Security Response Center
  - Új sérülékenységek fognak felbukkanni
  - Mit tegyünk, ha érintettek vagyunk?
  - Hogyan léphetnek velünk kapcsolatba, ha sérülékenységet találnak?

- **Incidens kezelési terv** → Incident Response Team
  - Mi legyen, ha a sérülékenységet nem felelősen hozták nyilvánosságra?
  - Mi van, ha megtámadják a rendszereinket?

Üzemeltetés

Fejlesztési életciklus

Sérülékenységek életciklusa



SDL után: Security Response

- **Security response terv** végrehajtása: security response folyamat

- Security Response Center:
  1. Sérülkenység riportok gyűjtése, válaszolás
  2. Riport elemzése → Fejlesztők el tudnak kezdeni dolgozni
  3. Kapcsolattartás megtalálókkal, felelős nyilvánosságra hozás bátorítása
  4. Security bulletin létrehozása
  5. Ügyfelek problémáinak és a sajtó figyelése

SDL után: Security Response

Media framework

The most severe vulnerability in this section could enable a remote attacker using a specially crafted file to execute arbitrary code within the context of a privileged process.

CVE	References	Type	Severity	Updated AOSP versions
CVE-2017-13248	A-70349612	RCE	Critical	6.0, 6.0.1, 7.0, 7.1.1, 7.1.2, 8.0, 8.1
CVE-2017-13249	A-70399408	RCE	Critical	6.0, 6.0.1, 7.0, 7.1.1, 7.1.2, 8.0, 8.1
CVE-2017-13250	A-71375536	RCE	Critical	6.0, 6.0.1, 7.0, 7.1.1, 7.1.2, 8.0, 8.1
CVE-2017-13251	A-69269702	EoP	Critical	6.0, 6.0.1, 7.0, 7.1.1, 7.1.2, 8.0, 8.1
CVE-2017-13252	A-70526702	EoP	High	8.0, 8.1
CVE-2017-13253	A-71389378	EoP	High	8.0, 8.1

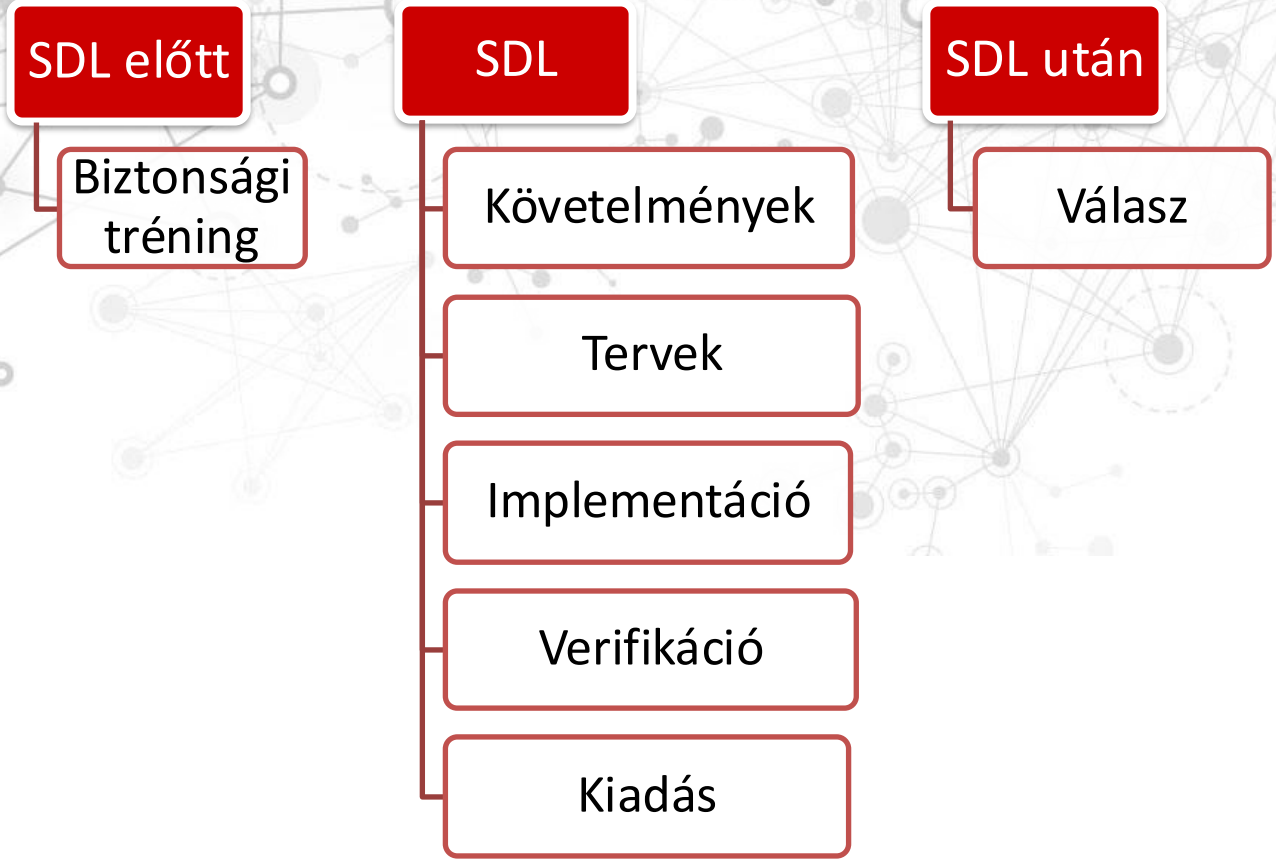
SDL után: Security Response

- **Security response terv** végrehajtása: security response folyamat
  - Fejlesztőcsapat:
    1. Javítás implementálása: eredeti és változatok is!
  - Tesztelő csapat:
    1. Eredeti problémák javításának a verifikálása (Security Team)
    2. Kompatibilitás ellenőrzése
  - Tanulságok dokumentálása

SDL után: Incidens kezelés

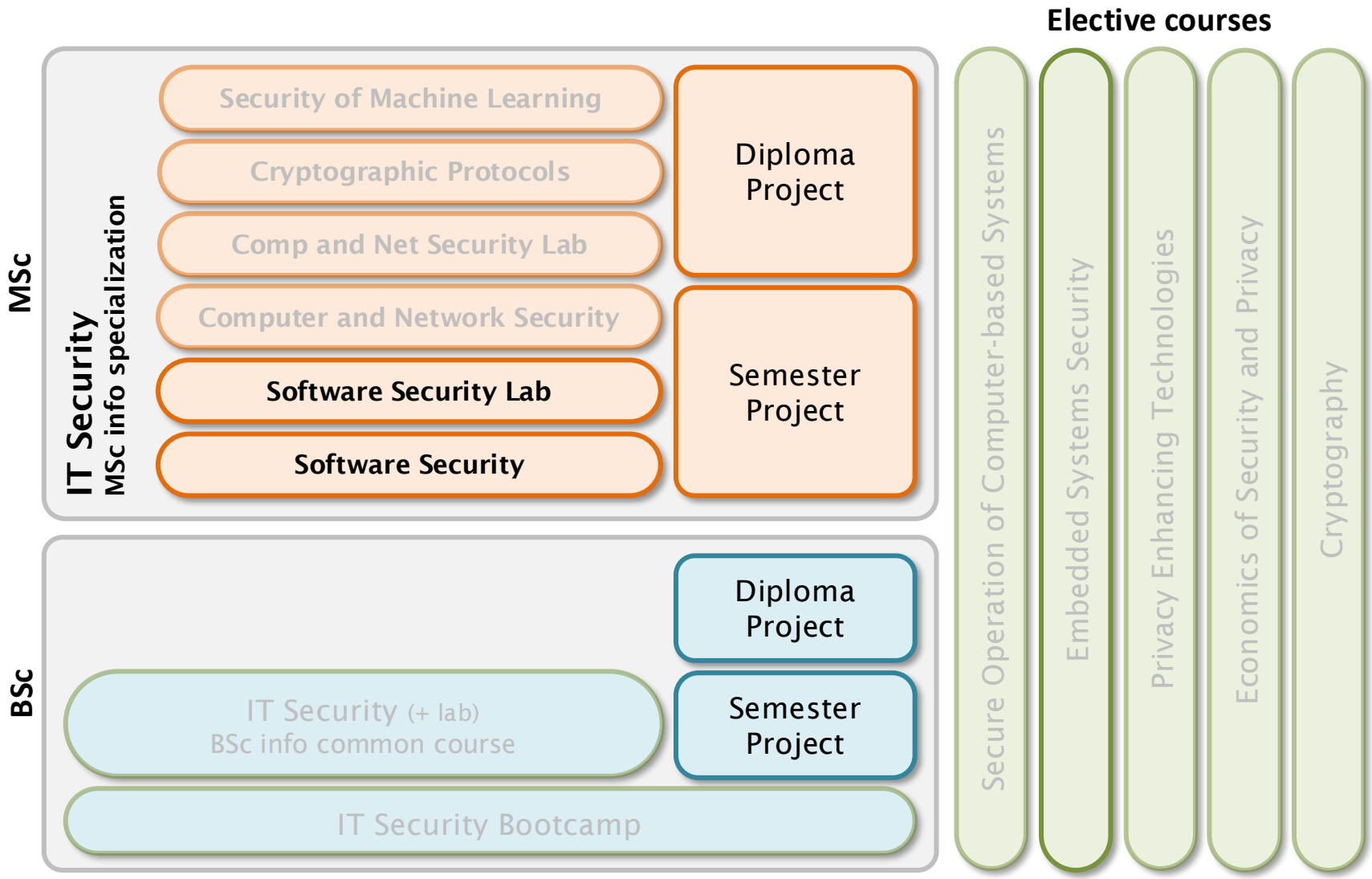
- Felelőtlen nyilvánosságra hozás, malware, stb. → **incidens kezelési plan**

- Incident response team:
  1. Környezet, ügyfelek, sajtó, stb. nyomon követése
  2. Security response team mobilizálása → security response plan
  3. Szituáció értékelése, tanácsadás, kerülő megoldások kommunikálása
  4. Információ és eszközök nyújtása a normál ügymenet helyreállításához



Építsétek be a biztonságot!

IT biztonság oktatási program (BSc, MSc)



more info: http://www.crysys.hu/education/

Ellenőrző kérdések

Ellenőrző kérdések

- Mi a CVE?
- Miért nehéz biztonságos szoftvert fejleszteni?
- Milyen fázisai vannak a Microsoft SDL-nek?
- Milyen feladatokat és szerepeket kell a személyzetnek ellátnia?
- Mit mond a "economy of mechanism" alapelv?
- Mit mond a "fail-safe defaults" alapelv?
- Mit mond a "complete mediation" alapelv?
- Mit mond a "separation of privilege" alapelv?
- Mit mond a "least privilege" alapelv?
- Mit mond a "open design" alapelv?

Ellenőrző kérdések

- Mit mond a "least common mechanism" alapelv?
- Mit mond a "psychological acceptability" alapelv?
- Mit értünk egy szoftver támadási felülete alatt?
- A szoftvere hibás bemenetet észlel. Hogyan reagáljon?
- Milyen információkra van szükség naplózáskor?
- Miért fontosak a naplóbejegyzések?
- Milyen típusú adatot nem szabad sose naplózni?
- Milyen előnyei és hátrányai vannak a statikus elemzésnek?
- Milyen előnyei és hátrányai vannak a dinamikus elemzésnek?
- Mi a fuzzing alapötlete? Mi az egyes komponensek feladata?
- Mi a célja a penetrációs tesztelésnek? Hogyan végzünk pentesztelést?

Ellenőrző kérdések

- Mi a különbség a security response és az incidens kezelés között?
- Mit jelent a sérülékenységek felelős nyilvánosságra hozatala?
- Hogyan néz ki egy sérülékenység életciklusa?

Háttéranyag

Háttéranyag

- Cybok: Secure Software Lifecycle Knowledge Area
  - https://www.cybok.org/media/downloads/Secure_Software_Lifecycle_v1.0.2.pdf
- Common Vulnerabilities and Exposures
  - https://cve.mitre.org/index.html
- Microsoft: Security Development Lifecycle
  - https://www.microsoft.com/en-us/SDL/process/requirements.aspx
- Steven M. Bellovin: Thinking Security: Stopping Next Year's Hackers. Addision-Wesley Professional, 2015

Háttéranyag

- US-CERT: Design Principles
  - https://www.us-cert.gov/bsi/articles/knowledge/principles/design-principles
- Stephen Northcutt, Jerry Shenk, Dave Shackleford, Tim Rosenberg, Raul Siles, Stev Mancini: Penetration Testing: Assessing Your Overall Security Before Attackers Do
  - https://www.sans.org/reading-room/whitepapers/analyst/penetration-testing-assessing-security-attackers-34635
- Ken Houghton: Vulnerabilities & Vulnerability Scanning
  - https://www.sans.org/reading-room/whitepapers/threats/vulnerabilities-vulnerability-scanning-1195